

CHAPTER 7

Building Software That Survives Scale

PART I – THEORY

1. Introduction

1.1 Aim

The aim of this chapter is to understand how Salesforce applications are designed to process large volumes of data efficiently. It introduces Governor Limits, bulkification, collections, and scalable Apex design principles to ensure applications remain reliable even when processing many records simultaneously.

1.2 Objectives

After completing this chapter, the learner will be able to:

- Understand why Salesforce developers must think about scalability.
 - Explain the importance of Governor Limits.
 - Understand bulkification in Apex.
 - Identify problems caused by SOQL inside loops.
 - Identify problems caused by DML inside loops.
 - Use Lists, Sets, and Maps effectively.
 - Design efficient Trigger architecture.
 - Build scalable Salesforce applications.
-

2. Introduction to Scalable Software

Enterprise applications are expected to process not only individual records but also hundreds or thousands of records during imports, integrations, or automated processes. Code that works correctly for a single record may fail when processing many records if it is not designed with scalability in mind. Salesforce developers therefore design applications to handle collections of records efficiently rather than processing each record individually.

3. Governor Limits

Definition

Governor Limits are limits imposed by Salesforce on database operations and system resources. Since Salesforce is a multi-tenant cloud platform, these limits ensure that all organizations share system resources fairly and efficiently.

Common Governor Limits

Some commonly encountered limits include:

- SOQL Queries – 100
- Records Retrieved – 50,000
- DML Statements – 150
- DML Records – 10,000
- CPU Time – 10,000 ms
- Heap Size – 6 MB

Importance

Governor Limits encourage developers to write optimized and efficient Apex code rather than relying on excessive database operations.

4. Bulkification

Definition

Bulkification is the process of designing Apex code so that it can process multiple records efficiently using collections instead of processing records individually. Bulkified code performs the same business logic safely whether Salesforce processes one record or two hundred records.

Benefits

- Improves application performance.
- Prevents Governor Limit exceptions.
- Supports bulk imports.
- Improves scalability.
- Encourages reusable code.

5. SOQL Inside Loops

Executing SOQL queries inside loops causes Salesforce to execute one query for every record being processed. When the number of records increases, the application may exceed the SOQL Governor Limit.

Example

```
for(Appointment__c appt : appointments){  
    // SOQL Query  
}
```

Best Practice

Collect all required record IDs first and execute a single SOQL query outside the loop.

6. DML Inside Loops

Executing DML operations inside loops performs multiple database transactions and may exceed the DML Governor Limit.

Example

```
for(Appointment__c appt : appointments){  
    update appt;  
}
```

Best Practice

Store records in a List and execute a single DML statement after processing all records.

7. Lists, Sets and Maps

List

A List stores multiple records while preserving their order.

Example

```
List<Appointment__c> appointments = new List<Appointment__c>();
```

Set

A Set stores only unique values and removes duplicates automatically.

Example

```
Set<Id> patientIds = new Set<Id>();
```

Map

A Map stores key-value pairs, allowing records to be retrieved quickly using their IDs.

Example

```
Map<Id, Patient__c> patientMap = new Map<Id, Patient__c>();
```

These collections are fundamental to designing bulk-safe Apex.

8. Bulk Processing Pattern

The recommended bulk-processing approach consists of the following steps:

1. Receive all records.
 2. Collect required IDs.
 3. Execute one SOQL query.
 4. Store records in Maps.
 5. Process records in memory.
 6. Collect records for update.
 7. Execute one DML statement.
-

9. Whiteboard Challenge (Hospital OPD Version)

Scenario

The Hospital OPD system receives 200 appointment records.

Solution

- Collect all Patient IDs.
- Collect all Doctor IDs.
- Retrieve all Patients using one SOQL query.
- Retrieve all Doctors using one SOQL query.
- Store records in Maps.
- Validate appointments.

- Update appointments using one DML operation.

This design minimizes database operations and follows Salesforce bulk-processing best practices. It adapts the chapter's whiteboard exercise to the Hospital OPD context.

10. Debug This

Problem 1

SOQL queries are written inside loops.

Solution

Move SOQL queries outside loops and retrieve all required records in a single query.

Problem 2

DML operations are executed inside loops.

Solution

Collect all modified records in a List and execute one DML operation.

Problem 3

Repeated queries retrieve the same information multiple times.

Solution

Store queried records in Maps and reuse them.

Problem 4

The application processes one record at a time.

Solution

Design the application to process collections of records using Lists, Sets, and Maps.

11. Interview Corner

Q1. What are Governor Limits?

Governor Limits are resource limits enforced by Salesforce to ensure efficient use of shared platform resources.

Q2. What is Bulkification?

Bulkification is the process of designing Apex code to process multiple records efficiently.

Q3. Why is SOQL inside loops dangerous?

Because it increases database queries and may exceed Salesforce Governor Limits.

Q4. Why is DML inside loops dangerous?

Because it creates unnecessary database operations and may exceed DML limits.

Q5. Why are Sets used?

Sets store unique values and remove duplicate IDs automatically.

Q6. Why are Maps used?

Maps provide fast access to records using a key, such as a record ID. These questions reflect the concepts emphasized in the chapter.

12. Sprint Retrospective

What worked well?

This chapter improved understanding of scalable Apex design and efficient database operations.

Most challenging concept

Understanding how collections such as Lists, Sets, and Maps improve performance during bulk processing.

Future Improvements

- Batch Apex
 - Queueable Apex
 - Scheduled Apex
 - Asynchronous Processing
-

13. Conclusion

This chapter introduced the fundamental principles of scalable Salesforce development. By understanding Governor Limits, bulkification, and collection-based processing, developers can build applications that efficiently handle large volumes of records. These concepts form the foundation for designing enterprise-level Salesforce solutions that remain reliable and maintainable as business data grows.

PART II – ENGINEERING SPRINT

1. Introduction

Aim

To implement a bulk-safe Hospital OPD Management System using Apex, SOQL, DML, Sets, Maps, and Trigger context variables while following Salesforce Governor Limits.

Objective

- Build bulk-safe Apex code.
 - Eliminate SOQL inside loops.
 - Eliminate DML inside loops.
 - Use Trigger.new and Trigger.oldMap.
 - Detect appointment completion.
 - Update related Prescription records.
 - Send notification messages.
-

2. Software Requirements

Software Used

- Salesforce CRM
- Visual Studio Code
- Salesforce CLI
- Trailhead Playground

Technologies

- Apex
- SOQL
- DML
- Apex Trigger

Objects Used

- Patient
- Doctor
- Appointment
- Prescription

3. Engineering Sprint 17 – Bulkifying Appointment Validation

Aim

To redesign the appointment validation process so that it efficiently handles multiple appointment records using bulk-safe Apex programming techniques.

Business Requirement

The Hospital OPD Management System should validate duplicate appointments even when multiple appointment records are inserted simultaneously without exceeding Salesforce Governor Limits.

Implementation

The duplicate appointment validation logic was redesigned by collecting Patient IDs, Doctor IDs, and Appointment Dates using Sets. A single SOQL query retrieves all matching appointments, and the validation is performed in memory. This approach eliminates SOQL queries inside loops and improves scalability.

Screenshot 1

Bulk-safe AppointmentTriggerHandler

```
1  // app > main > default > classes > AppointmentTriggerHandler.cls > ...
2  public class AppointmentTriggerHandler {
3
4      // Before Insert
5      public static void preventDuplicateAppointments(List<Appointment__c> newAppointments) {
6
7          Set<Id> patientIds = new Set<Id>();
8          Set<Id> doctorIds = new Set<Id>();
9          Set<Date> appointmentDates = new Set<Date>();
10
11         // Collect unique values
12         for (Appointment__c appt : newAppointments) {
13
14             if (appt.Patient__c != null)
15                 patientIds.add(appt.Patient__c);
16
17             if (appt.Doctor__c != null)
18                 doctorIds.add(appt.Doctor__c);
19
20             if (appt.Appointment_Date__c != null)
21                 appointmentDates.add(appt.Appointment_Date__c);
22         }
23
24         // One SOQL Query
25         List<Appointment__c> existingAppointments = [
26             SELECT Id,
27                 Patient__c,
28                 Doctor__c,
29                 Appointment_Date__c
30             FROM Appointment__c
31             WHERE Patient__c IN :patientIds
32             AND Doctor__c IN :doctorIds
33             AND Appointment_Date__c IN :appointmentDates
34         ];
35
36         // Validate duplicates
37         for (Appointment__c newAppt : newAppointments) {
38             for (Appointment__c existing : existingAppointments) {
39                 if (existing.Patient__c == newAppt.Patient__c &&
40                     existing.Doctor__c == newAppt.Doctor__c &&
41                     existing.Appointment_Date__c == newAppt.Appointment_Date__c) {
42                     newAppt.addError(
43                         'Duplicate appointment is not allowed for the same patient, doctor and appointment date.'
44                     );
45                 }
46             }
47         }
48     }
49 }
```


Screenshot 2

Bulk-safe Duplicate Validation

```
e-app > main > default > classes > AppointmentTriggerHandler.cs > ...
public class AppointmentTriggerHandler {
    public static void detectCompletedAppointments(
        List<Appointment_c> newAppointments,
        Map<Id, Appointment_c> oldMap
    ) {
        for (Appointment_c appt : newAppointments) {
            Appointment_c oldAppt = oldMap.get(appt.Id);

            if (oldAppt != null &&
                oldAppt.Status_c != 'Completed' &&
                appt.Status_c == 'Completed') {
                System.debug('Appointment completed: ' + appt.Name);
            }
        }
    }

    // After Update
    public static void updatePrescription(List<Appointment_c> updatedAppointments) {
        Set<Id> appointmentIds = new Set<Id>();

        for (Appointment_c appt : updatedAppointments) {
            if (appt.Status_c == 'Completed') {
                appointmentIds.add(appt.Id);

                // Notify Doctor
                NotificationService.notifyDoctor(appt);
            }
        }

        // One SQL Query
        List<Prescription_c> prescriptions = [
            SELECT Id,
                   Appointment_c,
                   Follow_up_Date_c
            FROM Prescription_c
            WHERE Appointment_c IN :appointmentIds
        ];

        List<Prescription_c> prescriptionsToUpdate = new List<Prescription_c>();

        // Update in memory
        for (Prescription_c p : prescriptions) {
            p.Follow_up_Date_c = Date.today().addDays(7);
            prescriptionsToUpdate.add(p);
        }
    }
}
```

Screenshot 3

Duplicate Appointment Validation

* = Required Information

Information

Appointment Name

Owner
Nikshitha Ramesh Midasala

Appointment Date

Status
--None--

Symptoms

Patient
Nikki Ramesh

Doctor
Dr. Kalyan

We hit a snag.

Review the errors on this page.

- Duplicate appointment is not allowed for the same patient, doctor and appointment date.

Cancel Save & New Save

4. Engineering Sprint 18 – Detecting Appointment Completion in Bulk

Aim

To detect appointment status changes efficiently using `Trigger.new` and `Trigger.oldMap` and update related Prescription records.

Business Requirement

Whenever an appointment status changes from **Scheduled** to **Completed**, the system should automatically:

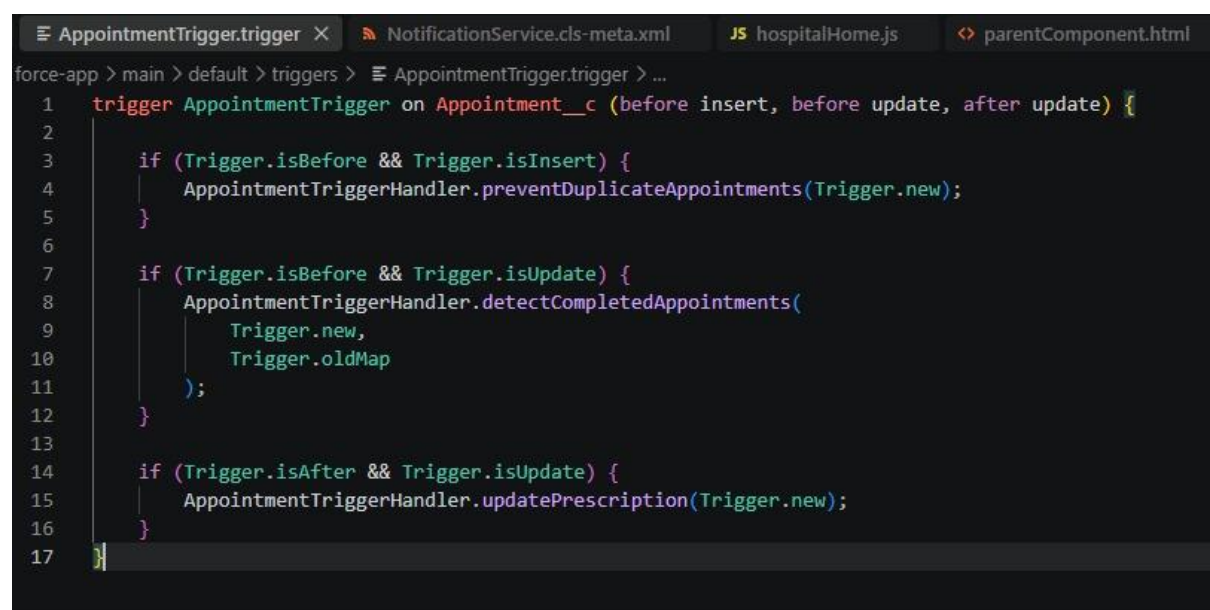
- Detect the status change.
 - Update the Prescription Follow-up Date.
 - Notify the doctor.
 - Perform all operations in a bulk-safe manner.
-

Implementation

A Before Update trigger compares the previous and current appointment status using `Trigger.oldMap` and `Trigger.new`. When an appointment becomes Completed, the related prescription is updated and the notification service is executed.

Screenshot 4

Appointment Trigger



```
force-app > main > default > triggers > AppointmentTrigger.trigger > ...
1  trigger AppointmentTrigger on Appointment__c (before insert, before update, after update) {
2
3      if (Trigger.isBefore && Trigger.isInsert) {
4          AppointmentTriggerHandler.preventDuplicateAppointments(Trigger.new);
5      }
6
7      if (Trigger.isBefore && Trigger.isUpdate) {
8          AppointmentTriggerHandler.detectCompletedAppointments(
9              Trigger.new,
10             Trigger.oldMap
11         );
12     }
13
14     if (Trigger.isAfter && Trigger.isUpdate) {
15         AppointmentTriggerHandler.updatePrescription(Trigger.new);
16     }
17 }
```

Screenshot 5

Appointment Record

Developer Edition

Search...

SalesHomeOpportunitiesLeadsTasksFilesAccountsContactsCampaignsDas

Appointment

APT-0009

RelatedDetails

Appointment Name

APT-0009

Appointment Date

06/08/2026

Status

Completed

Symptoms

Patient

Nikki Ramesh

Doctor

Dr. Kalyan

Created By

Nikshitha Ramesh Midasala, 06/08/2026, 3:03 pm

Owner

Nikshitha Ramesh Midasala

Last Modified By

Nikshitha Ramesh Midasala, 06/08/2026, 3:10 pm

Screenshot 6

Prescription Record

Developer Edition

Search...

SalesHomeOpportunitiesLeadsTasksFilesAccountsContactsCampaignsDa

Prescription

PRE-0010

RelatedDetails

Prescription Name

PRE-0010

Diagnosis

Medicines

Follow-up Date

13/08/2026

Appointment

APT-0009

Created By

Nikshitha Ramesh Midasala, 06/08/2026, 3:09 pm

Owner

Nikshitha Ramesh Midasala

Last Modified By

Nikshitha Ramesh Midasala, 06/08/2026, 3:10 pm

Screenshot 7

Debug Log

brave-otter-c53vtp-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File ▾ Edit ▾ Debug ▾ Test ▾ Workspace ▾ Help ▾ < >

Log executeAnonymous @8/6/2026, 3:13:29 PM

Execution Log

Timestamp	Event	Details
15:13:29:027	USER_DEBUG	[5] DEBUG Notification: Appointment APT-0009 has been completed successfully.

☐ This Frame ☐ Executable ☒ Debug Only ☐ Filter [Click here to filter the log](#)

Logs Tests Checkpoints Query Editor View State Progress Problems

User	Application	Operation	Time	Status
Nikshitha Ramesh Midasala	Browser	/aura	8/6/2026, 3:13:29 PM	Success
Nikshitha Ramesh Midasala	Unknown	common.api.soap.DirectSoap	8/6/2026, 3:13:29 PM	Success

5. Results

The Hospital OPD Management System successfully implemented bulk-safe Apex programming by:

- Preventing duplicate appointments.
- Eliminating SOQL inside loops.
- Eliminating DML inside loops.
- Detecting appointment status changes using Trigger.oldMap.
- Automatically updating Prescription records.
- Sending notification messages.
- Following Salesforce Governor Limits.

6. Debug This

Problem 1

SOQL inside loops increases database queries.

Solution

Execute one SOQL query outside the loop.

Problem 2

DML inside loops increases database operations.

Solution

Perform one DML operation after processing all records.

Problem 3

Processing one record at a time.

Solution

Use Lists, Sets, and Maps for bulk processing.

Problem 4

Business logic written directly inside Trigger.

Solution

Move business logic to Trigger Handler classes.

7. Interview Corner

Q1. What is Bulkification?

Bulkification is the process of writing Apex code that efficiently processes multiple records using collections.

Q2. Why should SOQL not be written inside loops?

It increases database queries and may exceed Salesforce Governor Limits.

Q3. Why should DML not be written inside loops?

It increases database operations and may exceed DML limits.

Q4. What is Trigger.oldMap?

Trigger.oldMap stores the previous version of records before an update operation.

Q5. What is Trigger.new?

Trigger.new contains the new version of records currently being processed.

Q6. Why are Sets used?

Sets store unique values and remove duplicates automatically.

Q7. Why are Maps used?

Maps provide fast access to records using their IDs.

Q8. What is the advantage of bulk-safe Apex?

Bulk-safe Apex processes multiple records efficiently while complying with Salesforce Governor Limits.

8. Sprint Retrospective

What worked well?

The Hospital OPD Management System successfully implemented scalable Apex code capable of processing multiple appointment records efficiently.

Most challenging task

Implementing bulk-safe validation and understanding Trigger.oldMap.

Future Improvements

- Email notifications
- Batch Apex
- Queueable Apex
- Scheduled Apex
- Dashboard reporting

9. Conclusion

The Engineering Sprint successfully enhanced the Hospital OPD Management System by implementing bulk-safe Apex programming techniques. The application now processes multiple appointment records efficiently using Sets, Maps, Trigger context variables, and optimized SOQL and DML operations. These improvements make the application scalable, maintainable, and compliant with Salesforce Governor Limits.